Computer Vision - Spring 26
Assignment 3

Instructors: Prof Makarand Tapaswi
and Prof Charu Sharma

Deadline: 04 April 2025 11:59 P.M.

Submisison for the assignment will be done via GitHub Classroom. Use the following to accept the assignment: https://classroom.github.com/a/2mK7Mv6s

1 Oriented Bounding Boxes through Faster RCNN [50 Marks]

You have been provided with an implementation of Faster RCNN in your GitHub Classroom. Your task is to understand the codebase and extend it to predict oriented bounding boxes instead of the standard axis-aligned bounding boxes.

For all the experiments that follow for this problem, you have to use this dataset here. Please visualize the dataset before starting. The training and inference files along with the dataloader are provided.

1.1 Understanding Faster RCNN (15 Marks)

Faster R-CNN consists of two main components: the **Region Proposal Network (RPN)** and the **ROI Head**.

The RPN is responsible for generating region proposals that may contain objects. To get started, for this subpart, you need to train the model for axis aligned bounding boxes only. Look at train.py for getting started.

For this subpart, you are expected to understand the code, and do a set of visualizations for different modules in the code. These visualizations will depend on a lot of hyperparameters. You are expected to understand the role of each of them, and do the same set of following visualizations for atleast 2 different sets of hyperparameters.

1. For the RPN, please understand the **Region ProposalNetwork** and the **RPNHead** class. Visualize the following for a fixed validation batch during training. (10 Marks)

   - **Visualizing predicted objectness map:** Visualize the evolution of objectness score heatmap for every feature map level coming from the backbone for this batch of images. Visualize it using a video over different validation iterations at different timesteps of training.

- **Visualizing predicted object proposals:** Create an animation demonstrating how object proposals change over iterations. Essentially for every image in this batch, visualize the bounding box proposals that ultimately go to the ROI Head, on the image itself, for successive validation iterations, and play that as an animation.
- **Visualizing bounding box assignments:** During the training of RPN, we sample both positive and negative anchors corresponding to the ground truth, that have a high and a low overlap with the ground truth boxes. Visualize the positive anchors with green color and negative anchors with red color on the image itself. While visualizing, limit the number of positive and negative anchors to 10.

2. For the ROI Heads, please understand the ROIHeads class. Visualize the following for the same validation batch during training. (5 Marks)

   - The ROI head ouptuts the classification scores and the bounding box regression terms, on taking the ROI of the object proposals coming from the RPN. Visualize the difference between the proposal given by the RPN and the final bounding box predicted predicted by the ROIHeads on the same image, at successive iterations of training by plotting both of these with a different color. Visualize the classification score as well on top of the predicted bounding box.

Note: You can make these visualizations by either storing the checkpoints explicity during training and then visualizing, or alternatively saving the visualized images during training itself. You can use OpenCV for making the animations and the videos.

1.2 Extending Faster R-CNN for Oriented Bounding Boxes (30 Marks)

For predicting oriented boxes instead of axis aligned boxes, you will need to modify the ROIHead class as well as the network head itself for predicting the angles along with the classification scores and the bounding boxes.

Note: You don't need to change the RPN class and can assume that the proposals themselves are axis aligned.

1. Modify the code to predict oriented bounding boxes:

   - Modify the dataset class for reading angles as well. Preprocess these and visualize the annotations for a fixed set of images.
   - Extend the head to predict an additional angle parameter along with class scores and bounding box regression.
   - Implement two different approaches for angle prediction:
     - Direct angle regression.
     - Multi-bin classification for discretized angle prediction. Here you will need to discretize the angles based on a fixed step size.

   Essentially, angle prediction will yield an additional loss term. Add this term to the existing loss terms by using a corresponding weight factor for this. You will need to tune the weight factor here.

2. Adjust the evaluation metrics:
   - Modify the Mean Average Precision (mAP) calculation to account for oriented bounding boxes.
   - Log Precision, Recall, and mAP throughout training.
3. Tune the hyperparameters of the model:
   - The loss weight for the angle prediction.
   - Try few bin sizes for multi-bin classification for the angle prediction. Compare the results with regression for angle prediction
   - Any other hyperparameters you want to experiment with.

1.3 Analysis (5 Marks)

1. Evaluate the models:
   - Compute mAP at different IoU thresholds for oriented bounding boxes.
   - Analyze Precision and Recall metrics.
   - Perform a qualitative analysis of detection results.
2. Visualize results:
   - For a subset of validation images, visualize predictions overlaid with ground truth.

2 Multi-Task Learning: Segmentation and Depth Estimation [30 Marks]

Use this dataset for training and evaluation. The images, segmentation masks and depth maps in the dataset are of size 256×256. Use validation split (25%) to evaluate the model's performance during training. For each of the following tasks, plot the combined and individual losses, mIoU (Mean Intersection over Union) for segmentation and RMSE(Root Mean Square Error) for depth estimation curves over the training process. Train each model for atleast 10 Epochs or until convergence. After training the model, evaluate its performance by reporting the mIoU and RMSE on the test set. Additionally, visualize few sample predictions from the test set for each task. Display the predicted segmentation mask and depth maps alongside the ground truth image and its corresponding masks side-by-side for comparison.

2.1 Vanilla Multi-Task U-Net [10 Marks]

Implement a U-Net architecture as shown in the following image.
Your task is to modify the standard Vanilla U-Net architecture to output two distinct predictions: a semantic segmentation mask and a depth map.

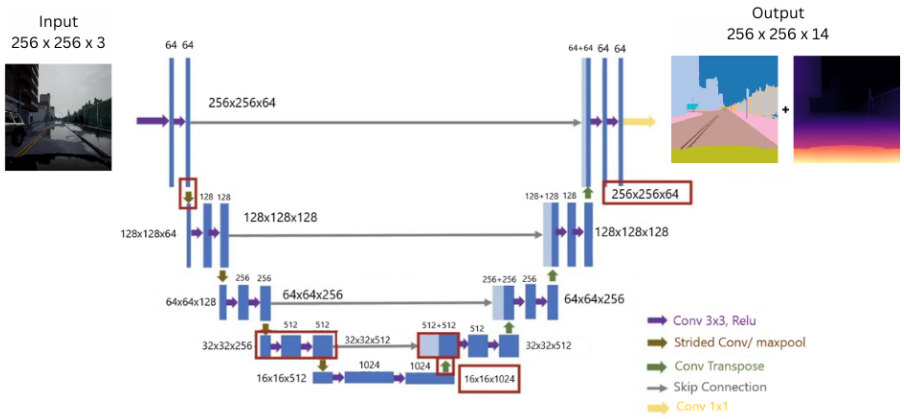

Figure 1: Modified U-Net architecture for simultaneous segmentation and depth estimation.

2.1.1 Combined Loss Function:

Use an appropriate combined loss function for segmentation and depth estimation and find a combination of the losses for reasonable predictions on both tasks.

2.1.2 Evaluation & Logging (Weights & Biases):

- Integrate **wandb** into your training loop to log your experiments.
- Track and plot the training loss, validation loss, validation mIoU, and validation Root Mean Squared Error (RMSE) for the depth task across all epochs.
- Provide a link to your wandb in your submission.
- Visualize at least 10 sample predictions from the test set side-by-side: Original Image, Ground Truth Mask, Predicted Mask, Ground Truth Depth, Predicted Depth.

2.2 Multi-Task U-Net without Skip Connections [10 Marks]

To understand the importance of spatial feature copying in multi-task networks, modify your architecture by removing all skip connections between the encoder and decoder paths.

- Train this modified architecture using the fixed loss hyperparameter values.
- Log your training process to wandb.
- Compare the final mIoU and depth RMSE with the Vanilla Multi-Task U-Net. Discuss the qualitative differences in your predictions, particularly around object boundaries in both the segmentation masks and depth maps.

2.3 Multi-Task U-Net with Residual Connections [10 Marks]

Deepen the feature extraction capabilities of your network by replacing the standard double convolutional blocks in both the encoder and decoder with **Residual Blocks**.

- Implement a residual block consisting of two 3×3 convolutional layers, BatchNorm, and ReLU activations, with a skip connection adding the input of the block to its output. Include a 1×1 convolution in the skip path if the channel dimensions change.
- Log your metrics to wandb.
- Provide a comparative analysis of this model against the Vanilla Multi-Task U-Net.

3 Learning on Point Clouds [20 Marks]

In this problem, you will implement a simplified PointNet-based model for 3D object classification and analyze what the network learns from unordered point sets. To focus purely on the symmetric aggregation function, you will implement a simplified version of PointNet that does not include the Spatial or Feature Transform Networks (T-Nets). Dataset for this task is available here.

3.1 Data Preprocessing and Vanilla PointNet (8 Marks)

1. **Data Preprocessing:** Implement a custom Dataset class that loads the `.ply` files. For every point cloud, apply suitable preprocessing steps like centering, normalization, etc...
2. **Architecture Implementation:** Implement the Vanilla PointNet architecture with the following specifications:
   - **Feature Extraction (MLP):** Apply a sequence of 1D Convolutions to project the input $3 \times N$ coordinates into higher-dimensional spaces.
   - **Global Aggregation:** Use a global `max` pooling operation across the point dimension to extract a single 1024-dimensional global feature vector.
   - **Classification Head:** Pass the global feature vector through fully connected layers for classifiation.
3. **Training:** Train the model using appropriate loss and optimizer. Log and plot the training loss, validation loss, training accuracy, and validation accuracy curves over the training process.

3.2 Permutation Invariance (4 Marks)

PointNet is theoretically designed to be strictly invariant to the ordering of input points. In this section, you will evaluate this property empirically.

- During inference on your test set, randomly permute the sequence of the points in each input point cloud batch before passing it to the network.
- Compare the model predictions of the permuted point clouds against the predictions of the originally ordered point clouds.

- Report the percentage of test samples for which the predicted class changes after permutation. Is the result what you expect?

3.3 Critical Point Analysis and Robustness (8 Marks)

The global feature vector in PointNet is obtained via a max pooling operation across the spatial dimension. Because of how max pooling works, only a sparse subset of the input points, called **critical points**, actually survive the operation to form the global representation.

1. **Extracting Critical Points:** For a given input point cloud, extract the features right before the max pooling layer to identify the unique points that contributed to the final 1024-dimensional vector.
2. **Visualization:** Visualize at least 5 sample point clouds from the test set. Create a side-by-side plot for each sample:
   - **Left:** The full original point cloud.
   - **Right:** The isolated critical points plotted in a distinct color , overlaid on top of a faint version of the original point cloud.
3. **Robustness Experiment:** PointNet can natively handle a variable number of input points.
   - Take a subset of your test data and extract the critical points for each sample.
   - Create a sparse point cloud containing only these critical points.
   - Pass only this sparse cloud through your trained network.
   - Report the classification accuracy on this modified, sparse dataset. Does the accuracy drop? Briefly explain why based on the network's architecture.

All the best!